
for

Chess Variation: Los Alamos Project

Completed by *Ethan Mearns*(u21501212)

I am a student at the University of Pretoria

Revision History

Name	Date	Reason For Changes	Version
EPE321	26/07/2023	Initial version	1.0
Ethan Mearns	17/08/2025	Update for current Project	1.1

Introduction

Purpose

The project outlined by this SRS document will be a chess platform, version 1.1 called Los Alamos which is a variation of chess played on a 6x6 board with no bishops, no castling and no en passant moves. In this SRS I will be describing the entire system including user management, gameplay, tournament features and the social features. The purpose of the chess platform is to introduce more people to the game of chess by easing them into it with an easier variation. The other aim is to foster a growing community of people that enjoy playing chess and give them an opportunity to make new friends with like-minded people and participate in tournaments together.

Project Scope

The platform will include a user login and account creation page as well as a home page with links to play against other players, add friends and join or create a tournament. The scope of this project is to introduce more people to the game of chess and help them to

learn the rules. The project is very beginner friendly and will include a tutorial on how to play Los Alamos chess. I would also like to potentially include a low-level bot to play against the user.

References

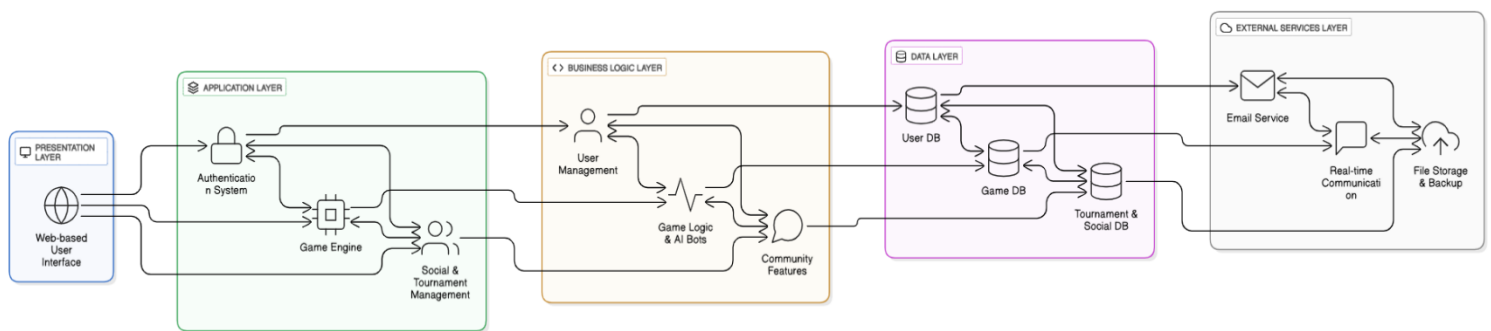
- Los Alamos Chess Rules: greenchess.net
- POPI Act (Protection of Personal Information Act) Compliance Guidelines

Overall Description

Product Perspective

Los Alamos will be a new self-contained product. Although there are plenty of chess platforms already available online, Los Alamos will be a tailored experience for beginners with tutorials on how to play. It will also contain a tournament, and friends feature to allow the community of chess within the university to grow. The system will be web-based and accessible through standard browsers, requiring no additional software installation, making it easily accessible to everyone.

Block Diagram



Product Functions

- **Authentication System**
 - Login page
 - Account creation page

- Forgot password functionality
- **Game Features**
 - Play online against other users
 - Play against friends
 - Play against AI bots (multiple difficulty levels)
 - Interactive Los Alamos tutorial
- **Tournament System**
 - View "My Tournaments"
 - Create tournaments
 - Join existing tournaments
 - Tournament bracket management
- **Social Features**
 - Friends list management
 - Friend requests and acceptance
 - Challenge friends to games
 - View friends' online status
- **Account Management**
 - Player game history (last 5 games)
 - Player ranking system
 - Profile settings
 - Account logout

User Classes and Characteristics

Guest Users (Unregistered)

- **Privilege Level:** Read-only access
- **Security Level:** Public access, no authentication required
- **Permissions:** View tutorial content, read platform information
- **Frequency:** One-time or infrequent visits

- **Technical expertise:** Basic computer literacy
- **Chess knowledge:** Variable
- **Primary needs:** Understanding platform features before registration
- **Restrictions:** Cannot play games, join tournaments, or access social features

Standard Users - Beginner Chess Players (University Students)

- **Privilege Level:** Basic authenticated user
- **Security Level:** Standard user authentication required
- **Permissions:**
 - Play games against other users and bots
 - Access tutorial system
 - Manage friend lists (send/receive requests)
 - Join tournaments (not create)
 - View personal game history and statistics
 - Update own profile information
- **Frequency:** Daily to weekly usage
- **Technical expertise:** Basic computer literacy
- **Chess knowledge:** Minimal to intermediate
- **Primary needs:** Learning chess rules, finding opponents, social interaction
- **Account Requirements:** Valid email for registration

Tournament Organizers (Elevated Users)

- **Privilege Level:** Enhanced user with tournament management rights
- **Security Level:** Standard authentication + organizer verification
- **Permissions:**
 - All standard user permissions
 - Create and manage tournaments
 - Set tournament rules and parameters
 - Send tournament invitations

- Access tournament statistics and reports
- Moderate tournament chat and behaviour
- **Frequency:** Weekly usage for tournament management
- **Technical expertise:** Intermediate computer skills
- **Chess knowledge:** Intermediate to advanced
- **Primary needs:** Creating and managing tournaments, community building
- **Verification Required:** Must be approved by system administrator

System Administrators

- **Privilege Level:** Full system access
- **Security Level:** Multi-factor authentication required
- **Permissions:**
 - All user and organizer permissions
 - User account management (suspend/ban users)
 - System configuration and maintenance
 - Database access and backup management
 - Security monitoring and incident response
 - Platform-wide announcements
 - Approve tournament organizer requests
- **Frequency:** Daily monitoring, as-needed maintenance
- **Technical expertise:** Advanced system administration skills
- **Chess knowledge:** Variable (not required for role)
- **Primary needs:** System security, performance monitoring, user support
- **Access Requirements:** Secure admin credentials, VPN access

Moderators (Optional)

- **Privilege Level:** Limited administrative rights
- **Security Level:** Standard authentication + moderator privileges
- **Permissions:**
 - Monitor chat and game conduct

- Issue warnings to users
- Temporary mute/suspend capabilities
- Access to user reports and complaints
- Escalate serious issues to administrators
- **Frequency:** Regular community monitoring
- **Technical expertise:** Basic to intermediate
- **Chess knowledge:** Intermediate (to understand game context)
- **Primary needs:** Community management, maintaining positive environment

Operating Environment

Client Side: Web browsers (Chrome, Firefox, Safari, Edge)

Server Side: Node.js runtime environment

Database: MongoDB or PostgreSQL (I know SQL so more likely of the two)

Operating Systems: Cross-platform web application. It is a web platform though and I need to investigate this.

Network: Internet connection required for real-time gameplay.

Hardware: Standard desktop/laptop computers, tablets, and mobile device.

Design and Implementation Constraints

The Los Alamos Chess Platform development is subject to several critical constraints that will influence design decisions and implementation approaches. The system must comply with University of Pretoria IT security policies and adhere to POPI Act requirements for data protection, ensuring all personal information is handled with care.

Development is constrained by an academic semester timeline, requiring careful scope management and prioritization of core features over advanced functionality. It is important to focus on the basics and not go off on tangents with grand ideas that are out of scope. As a student project, budget limitations restrict the use of premium hosting services, third-party APIs, and commercial software licenses, necessitating the use of open-source technologies and free-tier cloud services.

The platform must be accessible through standard web browsers without requiring plugins or specialized software installations, limiting the technology stack. Real-time gameplay functionality requires WebSocket or similar bidirectional technology.

Assumptions and Dependencies

Assumptions:

- Users have reliable internet connections
- Users have basic computer literacy
- University email system will be available for notifications
- Student demand exists for chess platform

Dependencies:

- Third-party email service integration
- Web hosting service availability
- Database hosting service
- Real-time communication library (Socket.io or similar) for live games
- Chess rule validation library

External Interface Requirements

User Interfaces

The user interface will follow modern web design with:

Login/Registration Pages:

- Clean, simple forms with clear field labels
- Password strength indicators
- Error message display areas
- Responsive design for mobile devices

Main Dashboard:

- Navigation menu with clear icons
- Quick access buttons for primary functions
- Online friends indicator
- Recent game history widget

Game Interface:

- 6x6 chess board with clear piece representation

- Move history panel
- Game timer display
- Chat interface for player communication
- Surrender/draw offer buttons

Tournament Interface:

- Tournament bracket visualization
- Registration forms
- Tournament status indicators
- Results display

All interfaces must be accessible and follow WCAG 2.1 guidelines.

Hardware Interfaces

The system is web-based and requires no specific hardware interfaces beyond standard computer input/output devices:

- Keyboard and mouse for user input
- Display monitor for visual output
- Network interface for internet connectivity

Software Interfaces

Database Interface:

- Connection to user account database
- Game state storage and retrieval
- Tournament data management
- Statistical data tracking

Email Service Interface:

- Integration with SMTP server for notifications
- Tournament invitations
- Friend request notifications
- Password reset functionality

Web Browser Interface:

- HTML5 compatibility
- JavaScript execution environment
- WebSocket support for real-time communication

Communications Interfaces

Real-time Game Communication:

- WebSocket connections for live gameplay
- JSON message format for game moves
- Automatic reconnection handling

Email Communications:

- SMTP protocol for outgoing emails
- SSL/TLS encryption for secure transmission
- Tournament notifications and friend requests

HTTP/HTTPS:

- RESTful API for client-server communication
- Secure authentication token transmission
- HTTPS encryption for all data transfer

System Features

4.1 User Authentication System

4.1.1 Description and Priority

Priority: High

Secure user registration, login, and account management system that allows users to create accounts, authenticate, and manage their profiles.

4.1.2 Stimulus/Response Sequences

1. User navigates to registration page
2. System displays registration form
3. User enters required information
4. System validates input and creates account

5. System sends confirmation email
6. User confirms account and can login

4.1.3 Functional Requirements

REQ-1: The system shall allow users to create accounts with username, email, and password. **REQ-2:** The system shall validate email addresses and ensure uniqueness. **REQ-3:** The system shall enforce password strength requirements (minimum 8 characters, mixed case, numbers). **REQ-4:** The system shall provide password reset functionality via email. **REQ-5:** The system shall maintain user sessions securely.

4.2 Los Alamos Game Engine

4.2.1 Description and Priority

Priority: High

Core gameplay functionality implementing Los Alamos chess rules, move validation, and game state management.

4.2.2 Stimulus/Response Sequences

1. User initiates game (vs bot, friend, or random opponent)
2. System creates game instance and 6x6 board
3. User makes move by clicking pieces
4. System validates move according to Los Alamos rules
5. System updates board state and switches turns
6. System checks for game end conditions
7. System records game result and updates statistics

4.2.3 Functional Requirements

REQ-6: The system shall implement a 6x6 chess board. **REQ-7:** The system shall exclude bishops from piece set. **REQ-8:** The system shall not implement castling or en passant moves. **REQ-9:** The system shall validate all moves according to Los Alamos rules. **REQ-10:** The system shall detect checkmate, stalemate, and draw conditions. **REQ-11:** The system shall support real-time multiplayer gameplay.

4.3 Tutorial System

4.3.1 Description and Priority

Priority: High

Interactive tutorial system teaching Los Alamos chess rules and basic strategies.

4.3.2 Stimulus/Response Sequences

1. User selects tutorial from main menu
2. System displays tutorial introduction
3. System guides user through piece movement
4. User practices moves on tutorial board
5. System provides feedback and progression
6. System marks tutorial completion

4.3.3 Functional Requirements

REQ-12: The system shall provide step-by-step piece movement tutorials. **REQ-13:** The system shall include practice scenarios for common positions. **REQ-14:** The system shall track tutorial progress. **REQ-15:** The system shall provide hints and guidance during tutorial.

4.4 Tournament Management

4.4.1 Description and Priority

Priority: Medium

Tournament creation, management, and participation system.

4.4.2 Stimulus/Response Sequences

1. Tournament organizer creates tournament
2. System generates tournament bracket
3. Players register for tournament
4. System manages tournament progression
5. System updates brackets after each game
6. System determines and announces winners

4.4.3 Functional Requirements

REQ-16: The system shall allow users to create tournaments with configurable settings. **REQ-17:** The system shall manage tournament brackets automatically. **REQ-18:** The system shall send tournament notifications via email. **REQ-19:** The system shall track tournament statistics and winners.

4.5 Social Features

4.5.1 Description and Priority

Priority: Medium

Friend management, social interaction, and community features.

4.5.2 Stimulus/Response Sequences

1. User searches for friends by username
2. System displays search results
3. User sends friend request
4. System notifies recipient via email
5. Recipient accepts/declines request
6. System updates friend lists

4.5.3 Functional Requirements

REQ-20: The system shall allow users to send and receive friend requests. **REQ-21:** The system shall display online status of friends. **REQ-22:** The system shall allow friends to challenge each other to games. **REQ-23:** The system shall provide basic chat functionality during games.

4.6 AI Bot System

4.6.1 Description and Priority

Priority: Low

AI opponents with multiple difficulty levels for practice games.

4.6.2 Stimulus/Response Sequences

1. User selects "Play vs Bot"
2. System presents difficulty options
3. User selects difficulty level
4. System initializes AI with appropriate parameters
5. Game proceeds with AI making moves automatically
6. System records game results

4.6.3 Functional Requirements

REQ-24: The system shall provide 3-4 AI difficulty levels. **REQ-25:** The system shall make AI moves within 5 seconds. **REQ-26:** The system shall use appropriate algorithms for move selection.

Other Nonfunctional Requirements

Performance Requirements

Response Time: Page loads shall complete within 3 seconds under normal network conditions

Game Moves: Move validation and board updates shall occur within 1 second

Number of Users at one given time: System shall support minimum 50 simultaneous players

Database Queries: Database operations shall complete within 2 seconds

AI Move Time: Bot moves shall be calculated within 5 seconds maximum

Security Requirements

Authentication: All user sessions must be securely authenticated using JWT tokens

Password Storage: Passwords must be hashed using bcrypt with minimum 12 salt rounds

Data Transmission: All client-server communication must use HTTPS encryption

Input Validation: All user inputs must be validated and sanitized server-side

POPI Compliance: Personal data handling must comply with Protection of Personal Information Act

Session Management: User sessions must expire after 24 hours of inactivity

Software Quality Attributes

Usability:

- New users should be able to complete tutorial within 15 minutes
- Navigation should be intuitive with maximum 3 clicks to reach any feature
- Interface should be responsive across desktop and mobile devices

Reliability:

- System uptime shall be minimum 95%
- Game data shall not be lost due to network interruptions
- Automatic game state recovery for disconnected players

Maintainability:

- Code shall follow established coding standards
- System shall use modular architecture for easy updates
- Comprehensive logging for debugging purposes

Scalability:

- Architecture shall support horizontal scaling
- Database design shall accommodate growing user base
- Caching mechanisms for improved performance

Other Requirements

6.1 Legal Requirements

- Compliance with University of Pretoria IT policies
- POPI Act compliance for personal data protection
- Terms of service and privacy policy implementation

6.2 Internationalization Requirements

- Support for English language (primary)
- Potential for Afrikaans language support in future versions

6.3 Database Requirements

- User data retention for minimum 1 year
- Game history storage for last 5 games per user
- Tournament data archival for statistical purposes
- Regular backup procedures every 24 hours

Appendix A: Glossary

Los Alamos Chess: A chess variant played on a 6x6 board without bishops, castling, or en passant moves

SRS: Software Requirements Specification

JWT: JSON Web Token - a security token format

POPI: Protection of Personal Information Act (South African privacy law)

WebSocket: Communication protocol for real-time web applications

API: Application Programming Interface

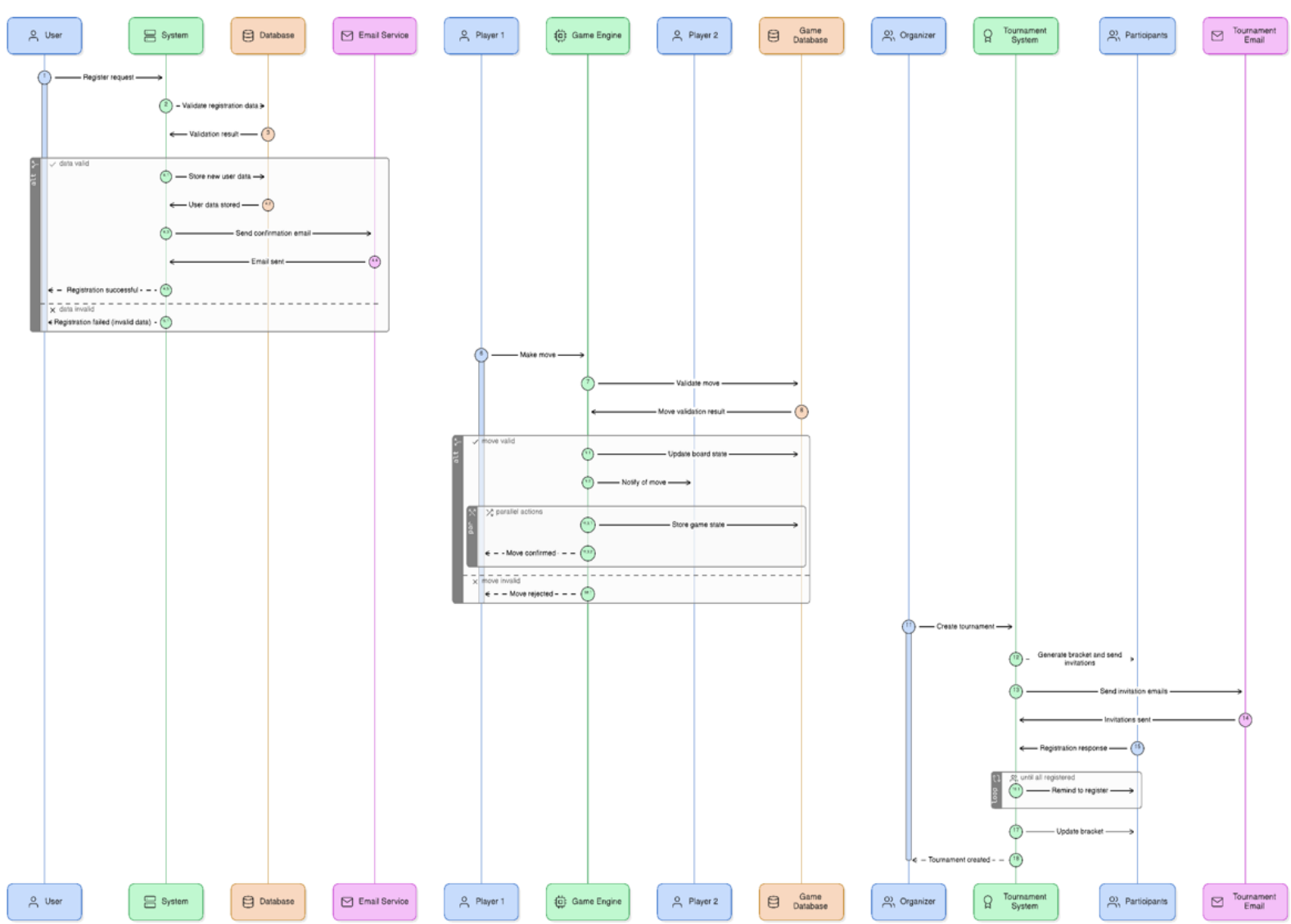
HTTPS: Hypertext Transfer Protocol Secure

UI: User Interface

Bot: Artificial Intelligence opponent

ELO Rating: Chess rating system for player skill assessment

Appendix B: Swimlane



Some Other Small Comments

Priority Implementation Order:

1. **Phase 1:** User authentication, basic game engine, tutorial
2. **Phase 2:** Multiplayer functionality, friend system
3. **Phase 3:** Tournament system, AI bots
4. **Phase 4:** Advanced features, statistics, optimizations

Risk Mitigation:

- **Technical Risk:** Complex real-time multiplayer - Mitigate with proven WebSocket libraries
- **Scope Risk:** Feature creep - Maintain strict priority adherence
- **Security Risk:** Data breaches - Implement security best practices from start
- **User Adoption Risk:** Low engagement - Focus on tutorial quality and user experience